DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go

BCSE308P – COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Science and Engineering (Cyber Security)

in

Department of Cybersecurity and Intelligent Systems

by

Student Name(s)

Register Number(s)



School of Computer Science Engineering and Information Systems [SCORE]

VIT, Vellore

November, 2026

November, 2026

DECLARATION

I hereby declare that the BCSE308P – Course Project entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by me, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of Faculty Guide Name, Designation, SCORE, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the **BCSE308P – Course Project** entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by Student Name & Reg. No., SCORE, VIT, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems, is a record of bonafide work carried out under my supervision during the period dd.mm.yyyy to dd.mm.yyyy, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE – Guide

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Faculty Guide Name, Designation, School of Computer Science Engineering and Information Systems (SCORE), VIT, Vellore, for the guidance, encouragement and valuable suggestions given throughout this project.

I thank the Dean of SCORE, the Head of the Department of Cybersecurity and Intelligent Systems and all the faculty members of the department for providing the knowledge, facilities and environment needed to carry out this work. I am grateful to the management of Vellore Institute of Technology for the opportunity to undertake this course project.

I also acknowledge the authors of the Internet standards and of the open-source tools on which this project builds, and the operators of the public DNS infrastructure whose servers answered the many queries sent during testing.

Finally, I thank my family and friends for their constant support and encouragement.

Student Name

EXECUTIVE SUMMARY

The Domain Name System (DNS) turns names into addresses for almost every Internet connection, which makes it a prime target for spoofing and cache poisoning. Yet the recursive resolver, the component that walks the DNS tree and decides which answers to trust, is usually hidden inside large production systems or libraries.

This project presents DNSR, an iterative DNS resolver written from scratch in Go using only the standard library. It encodes and decodes the RFC 1035 format by hand, resolves names from the root servers with QNAME minimisation, follows referrals, glue, glueless delegations and CNAMEs, and falls back to TCP. It rejects forged replies using random IDs, ports and letter case, rejects out-of-bailiwick data, validates DNSSEC signatures from the root trust anchors, and bounds its caches. It runs as a traced command-line tool or as an access-controlled, rate-limited server over UDP, TCP and DNS over TLS 1.3.

All 32 offline tests against a simulated hierarchy with signed and hostile servers pass. Live, DNSR matched Google Public DNS for 15 domains (median 557 ms cold, under 1 ms cached), validated seven signed domains as secure and rejected a mis-signed one, and survived 9.2 million fuzzing inputs.

CONTENTS

		Page No.
	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
	Symbols and Notations	xii
1	**INTRODUCTION**	1
	1.1 Objective	1
	1.2 Motivation	2
	1.3 Background	2
2	**PROJECT DESCRIPTION AND GOALS**	3
	2.1 Literature Review	3
	2.2 Gap Identification and Problem Statement	3
	2.3 Goals and Scope	3
3	**TECHNICAL SPECIFICATION**	5
	3.1 Requirements	5
	3.2 Environment and Protocol Details	5
4	**DESIGN APPROACH AND DETAILS**	7
	4.1 Design Approach / Materials and Methods	7
	4.2 Codes and Standards	11
	4.3 Constraints, Alternatives and Trade-offs	11
5	**SCHEDULE, TASKS AND MILESTONES**	12
6	**PROJECT DEMONSTRATION**	13
	6.1 Sample Codes	13
	6.2 Sample Screen Shots	14
7	**COST ANALYSIS / RESULT AND DISCUSSION**	16
	7.1 Functional Tests	16

7.2 Live Resolution and Caching 16
7.3 DNSSEC Validation 17
7.4 Server Mode, Security and Robustness 18
7.5 Security Review and Goals 19
7.6 Cost Analysis and Discussion 19
8 SUMMARY 20
9 REFERENCES 21
Appendix A – Source Code and Usage 24

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Recursive Query from a Stub and Iterative Queries from the Resolver	2
3.1	Layout of a DNS Message and Its 12-Byte Header	6
4.1	System Architecture of DNSR	7
4.2	Bailiwick Decisions for Replies from a .com Server	9
4.3	Chain of Trust from the Root Trust Anchor to an Answer	10
4.4	Structure of the Recursive Server Mode	11
5.1	Gantt Chart of the Project Schedule	12
6.1	Resolving www.github.com with QNAME Minimisation	14
6.2	DNSSEC Chain of Trust for cloudflare.com	15
6.3	An Independent Client Using UDP, TCP and DNS over TLS	15
6.4	Refused Client and TLS Version Enforcement	15
7.1	Cold-Cache Resolution Time per Domain	17
7.2	Probability of a Successful Blind Spoof	18

LIST OF TABLES

Table No.	Title	Page No.
2.1	Project Goals	3
3.1	Main Functional and Security Requirements	5
3.2	Environment	5
4.1	Modules	7
4.2	Main Design Decisions	11
5.1	Milestones	12
7.1	Offline Test Results	16
7.2	Effect of Caching (ms; reduction in brackets)	17
7.3	DNSSEC Results (Median of Three Cold-Cache Runs)	17
7.4	Robustness and Performance	19
7.5	Achievement of Goals	19

LIST OF ABBREVIATIONS

AA	Authoritative Answer (header flag)
AAAA	IPv6 address record
AD	Authenticated Data (header flag)
API	Application Programming Interface
CD	Checking Disabled (header flag)
CDN	Content Delivery Network
CLI	Command-Line Interface
CNAME	Canonical Name record
DNS	Domain Name System
DNSKEY	DNS public key record
DNSSEC	DNS Security Extensions
DO	DNSSEC OK (EDNS flag)
DoH	DNS over HTTPS
DoT	DNS over TLS
DS	Delegation Signer record
ECDSA	Elliptic Curve Digital Signature Algorithm
EDNS(0)	Extension Mechanisms for DNS, version 0
FQDN	Fully Qualified Domain Name
IANA	Internet Assigned Numbers Authority
ID	Transaction Identifier
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISP	Internet Service Provider
KSK	Key-Signing Key
mDNS	Multicast DNS
MTU	Maximum Transmission Unit
MX	Mail Exchanger record
NODATA	Name exists but has no records of the requested type
NS	Name Server record
NSEC, NSEC3	Next Secure records (authenticated denial of existence)
NXDOMAIN	Non-Existent Domain (response code 3)
OPT	EDNS(0) option pseudo-record
OS	Operating System
PTR	Pointer record (reverse lookup)

QNAME	Query Name
QR	Query/Response (header flag)
RA	Recursion Available (header flag)
RCODE	Response Code
RD	Recursion Desired (header flag)
RDATA	Resource Record Data
RFC	Request for Comments
RR	Resource Record
RRset	Resource Record Set
RRSIG	Resource Record Signature
RSA	Rivest–Shamir–Adleman public-key algorithm
RTT	Round-Trip Time
SERVFAIL	Server Failure (response code 2)
SLD	Second-Level Domain
SOA	Start of Authority record
TC	Truncation (header flag)
TCP	Transmission Control Protocol
TLD	Top-Level Domain
TLS	Transport Layer Security
TTL	Time To Live
TXT	Text record
UDP	User Datagram Protocol
ZSK	Zone-Signing Key

SYMBOLS AND NOTATIONS

P	Probability that a blind spoofing attempt succeeds
N	Number of equally likely (ID, port, letter-case) combinations
k	Number of forged replies that arrive before the genuine reply
Rport	Size of the ephemeral source-port range
ℓ	Number of letters in the query name (0x20 bits)
TTL	Time to live of a record (seconds)

CHAPTER 1

INTRODUCTION

The Domain Name System (DNS) turns names such as `www.github.com` into the IP addresses that the network routes on, and almost every Internet connection starts with a DNS lookup. If an attacker can change the answer, users are silently redirected without any encryption being broken. The component that decides which answers to trust is the *recursive resolver*, which walks the DNS tree from the root servers down to the server responsible for a name. In practice it is hidden inside large systems such as BIND and Unbound or inside an operating system's library, so the rules that attackers try to break are rarely seen or tested.

This project, **DNSR**, is a recursive resolver written from scratch in Go using only the standard library. Every byte of the RFC 1035 message format [1], [2] is encoded and decoded by hand. DNSR resolves names from the 13 IANA root servers, follows referrals, glue and aliases, defends against spoofing and cache poisoning, validates DNSSEC signatures, minimises the names it reveals, and prints a readable trace of each step. It runs as a command-line tool or as a recursive server over UDP, TCP and DNS over TLS.

1.1 Objective

- ✓ Implement an RFC 1035 encoder and decoder with name compression, EDNS(0) [3] and the A, AAAA, NS, CNAME, SOA, PTR, MX, TXT, DS, DNSKEY and RRSIG record types.
- ✓ Implement iterative resolution from the root: referrals, glue, glueless delegations, CNAME chasing, NXDOMAIN and NODATA, and TCP fallback.
- ✓ Defend against spoofing and poisoning: random IDs, ports and letter case, strict reply matching and bailiwick checks; and validate DNSSEC from the root trust anchors.
- ✓ Protect privacy and the service: QNAME minimisation, DNS over TLS 1.3, an access list, rate limiting and bounded caches.
- ✓ Verify the system offline against hostile and signed test servers and live against the Internet.

1.2 Motivation

In 2008 Kaminsky showed that an off-path attacker could poison almost any resolver within seconds by racing forged referrals with forged glue against genuine replies [4]. The defences, unpredictable queries and accepting only data a server is entitled to give, are codified in RFC 5452 and RFC 2181 [5], [6], yet weak checks have been found in deployed resolvers [7] and side channels have revived the attack [8]. Building a resolver that must enforce these checks, and seeing exactly where each one sits, is an effective way to understand them.

1.3 Background

The DNS namespace is a tree divided into zones. The root zone delegates top-level domains such as `com.`, which delegate second-level domains such as `github.com.`, using NS records. A stub resolver in the operating system sends its question to a recursive resolver, which asks the root, TLD and authoritative servers in turn with non-recursive queries, as shown in Fig. 1.1, and caches the answer for its time to live (TTL). Classic DNS runs over unauthenticated UDP: a reply is accepted if it carries the right 16-bit ID and answers the right question, which is what blind spoofing attacks try to guess.

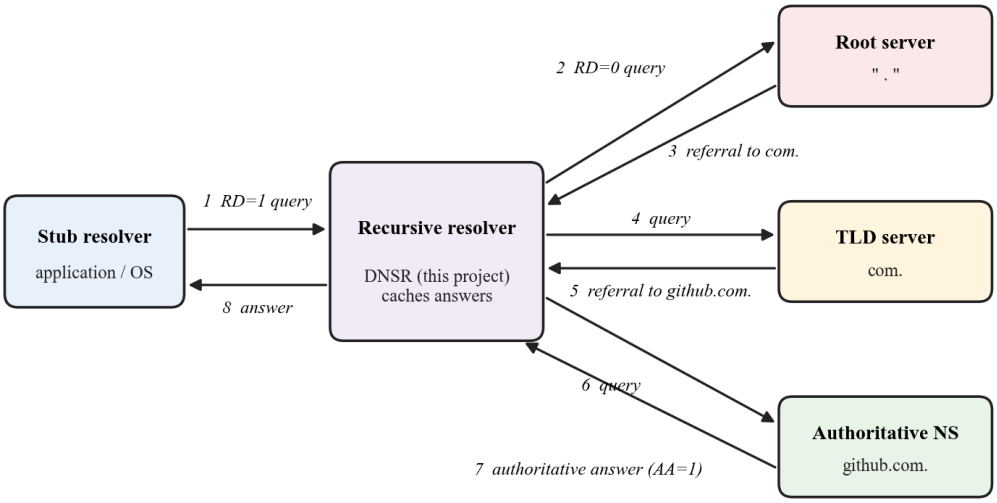


Fig. 1.1 Recursive Query from a Stub and Iterative Queries from the Resolver

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

RFC 1034 and 1035 [1], [2] define the namespace, the resolver algorithm and the message format; RFC 2181 [6] adds the ranking of data that underlies bailiwick checks. EDNS(0) [3] allows larger UDP replies, DNS Flag Day 2020 [9] recommends a 1232-byte buffer, and RFC 7766 [10] makes TCP fallback mandatory. RFC 5452 [5] requires random IDs and ports and full reply matching; Dagon et al. [11] add entropy by randomising letter case (0x20). Son and Shmatikov [7] and the SAD DNS attack [8] show that poisoning remains possible when checks are weak, and the NXNSAttack [12] shows that referral handling can be abused for amplification. Jung et al. [13] found that caching delegations is what most reduces resolver load. DNSSEC [14], [15], [16] authenticates data with signatures, QNAME minimisation [17] hides query names from upstream servers, and DNS over TLS [18] encrypts the client connection.

Production resolvers such as BIND [19] and Unbound [20] implement all of this but are hundreds of thousands of lines of C, Go's `net.Resolver` only forwards to an upstream resolver, and libraries such as `miekg/dns` [21] hide the wire format.

2.2 Gap Identification and Problem Statement

Existing tools are either opaque (stubs and libraries), too large to study (production resolvers) or observation-only (`dig +trace`), and the security checks that stop cache poisoning are rarely made explicit or tested repeatably. The problem addressed is therefore to build a compact, transparent and verifiable recursive resolver that implements the protocol from first principles, shows every step, enforces modern defences including DNSSEC, and can be tested offline against hostile servers.

2.3 Goals and Scope

Table 2.1 Project Goals

ID	Goal	Measure of success
G1	Correct codec and resolution	All record types parsed; 15 live domains match a public resolver
G2	Spoofing and poisoning defences	Forged replies, poisoned glue and out-of-bailiwick referrals rejected in tests

ID	Goal	Measure of success
G3	DNSSEC validation	Signed domains secure, unsigned insecure, broken bogus
G4	Privacy	Minimised upstream names; DNS over TLS for clients
G5	Performance	Measurable gain from caching
G6	Resilient server	Access list, rate limit and resource limits verified
G7	Robustness	No crash under large-scale fuzzing

Out of scope: proofs of non-existence in DNSSEC (NSEC and NSEC3), IPv6 transport to name servers, negative caching and DNS over HTTPS.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

Table 3.1 Main Functional and Security Requirements

ID	Requirement
FR1	Resolve a name and record type iteratively from the IANA root hints, without an upstream resolver
FR2	Follow referrals and glue; resolve glueless name servers; follow CNAMEs; separate NXDOMAIN from NODATA; retry over TCP on TC = 1
FR3	Cache answers and delegations; print a trace or only the answer
FR4	Serve clients over UDP, TCP and DNS over TLS; validate with DNSSEC on request; minimise query names by default
SR1	Random ID (cryptographic source), random source port and 0x20 letter case; accept a reply only if ID, question and case match
SR2	Accept referrals and glue only within the bailiwick of the replying server
SR3	Validate DNSSEC from the IANA root trust anchors (KSK-2017, KSK-2024)
SR4	Bound parsing (pointer jumps, name length), work (hops, depth, at most 3 glueless lookups), TTLs (1 day) and caches (10,000 entries)
SR5	Server: access list (loopback by default), per-client rate limit, at most 256 queries in flight, TLS 1.3 only

3.2 Environment and Protocol Details

Table 3.2 Environment

Item	Specification
Hardware and OS	Intel Core i7-13650HX; Windows 11 (64-bit)
Language	Go 1.25 module, standard library only (built with go1.27.0)
Validation tools	dnspython 2.7.0 [22]; Google Public DNS (8.8.8.8) as reference

A DNS message has a 12-byte header (ID, flags QR, AA, TC, RD, RA, AD, CD and RCODE, and four counts) followed by the question, answer, authority and additional sections, as shown in Fig. 3.1. Names are length-prefixed labels and may be compressed with 14-bit pointers to earlier occurrences; DNSR follows at most 64 pointers and rejects names over 255 bytes. Each query carries an EDNS(0) OPT record advertising 1232 bytes and, for DNSSEC, the DO bit.

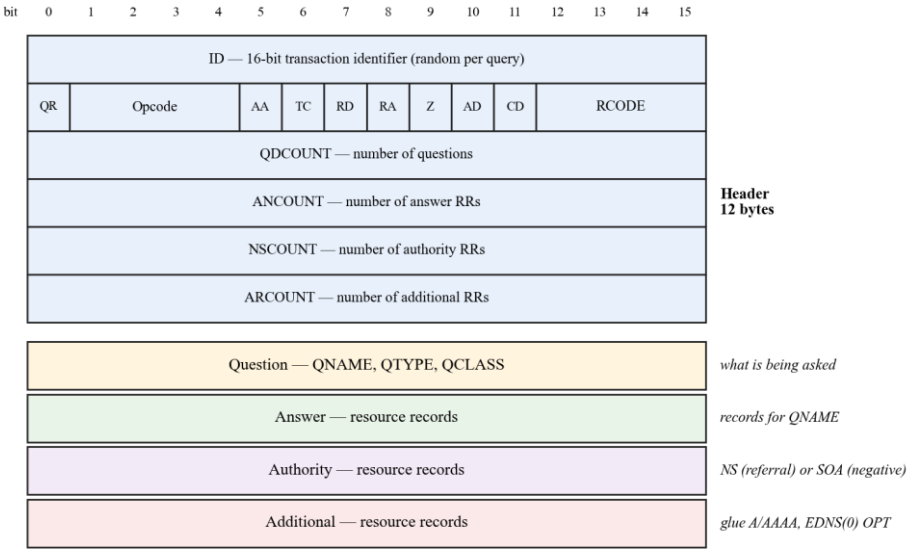


Fig. 3.1 Layout of a DNS Message and Its 12-Byte Header

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Design Approach / Materials and Methods

The project followed an incremental, test-driven method: each feature was specified from its RFC, implemented, and covered by a test against a fake DNS hierarchy before the next layer was added. Fig. 4.1 shows the resulting layered architecture, and Table 4.1 the modules.

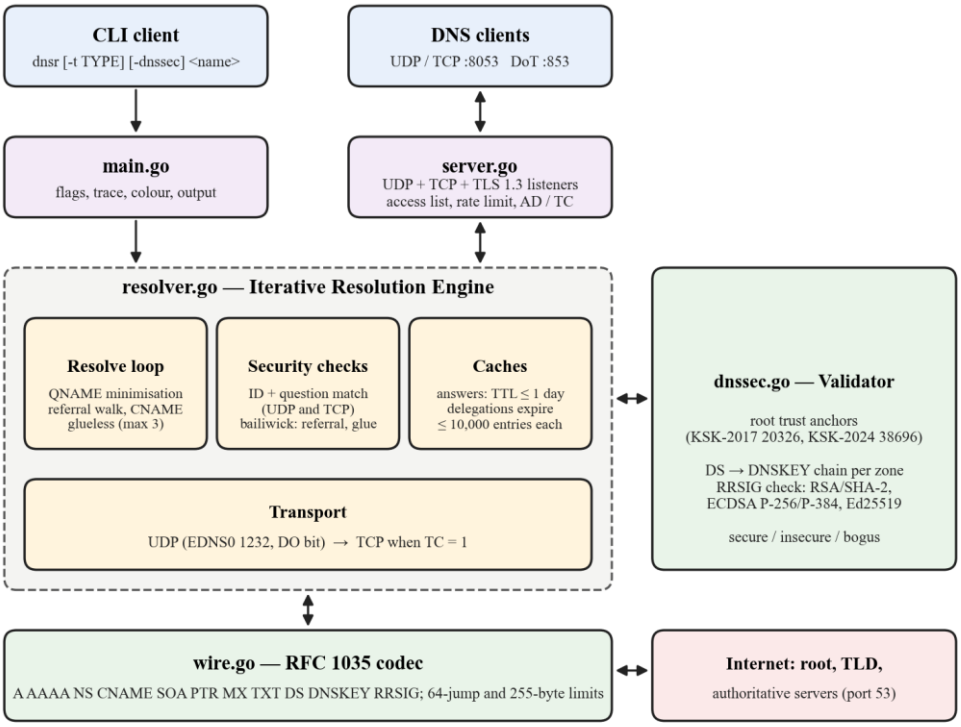


Fig. 4.1 System Architecture of DNSR

Table 4.1 Modules

File (lines)	Responsibility
wire.go (363)	Message types; encoding of queries; bounded decoding of names and records, including DNSSEC types
resolver.go (719)	Iterative resolution with QNAME minimisation; reply matching with 0x20; bailiwick checks; caches; TCP fallback
dnssec.go (527)	Trust anchors; RRSIG, DNSKEY, DS parsing; canonical form; signature checks; chain of trust
server.go (459)	UDP, TCP and DNS over TLS listeners; access list; rate limit; AD, SERVFAIL and TC handling
main.go (137)	Command-line flags and output

File (lines)	Responsibility
Tests (900)	Fake hierarchy with signed and hostile zones; 32 tests

Resolution algorithm

`resolve` checks the cache, starts at the deepest known zone and loops. While the target is more than one label below the current zone it sends a minimised NS probe; otherwise it sends the real question. It returns answers (following CNAMEs by restarting for the target), NXDOMAIN or NODATA, or follows a referral after the bailiwick check, resolving at most three name-server addresses itself when there is no glue. Two counters (40 hops, depth 8) guarantee termination.

Security design

For each query DNSR draws a 16-bit ID from `crypto/rand`, sends it from a new connected UDP socket (a random ephemeral port) and randomises the case of every letter of the name. A reply is accepted only if its ID, question and exact letter case match; anything else is ignored until the deadline. If k forged replies each guess among N combinations, the chance that one is accepted is given by Eqn. 4.1.

$$P = 1 - \left(1 - \frac{1}{N}\right)^{k} \approx \frac{k}{N} \tag{4.1}$$

With the ID alone, $N = 2^{16}$; with the 16,384 Windows ephemeral ports [23] and ℓ letters of 0x20 it grows as in Eqn. 4.2, so a 13-letter name needs about 6×10^{12} forged packets for an even chance. Servers that do not preserve case are detected and asked again without 0x20.

$$N = 2^{16} \times R_{port} \times 2^{\ell} \tag{4.2}$$

Even genuine replies may only speak for their own zone (Fig. 4.2): a referral must move strictly down towards the queried name, and glue is used only for referred name servers inside the sending server's zone, which defeats Kaminsky's forged-glue technique. Only answer records for the exact queried name are cached, and CNAME targets are always resolved afresh.

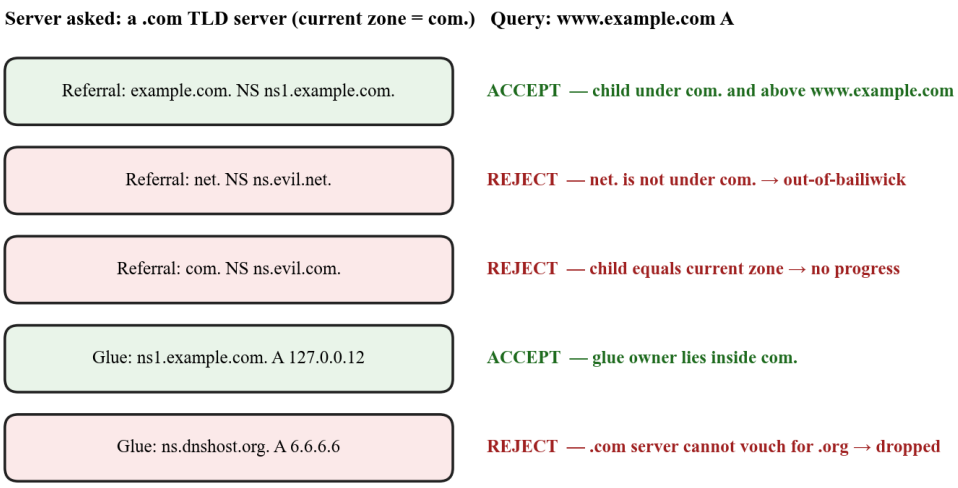


Fig. 4.2 Bailiwick Decisions for Replies from a .com Server

QNAME minimisation and DNSSEC

With QNAME minimisation (RFC 9156 [17]) the root is asked only `com. NS` and the TLD only `github.com. NS`; only the final server sees the full question. Unexpected replies to a probe make DNSR fall back to the full name. With `-dnssec`, every answer must be signed by a key that chains to the root trust anchors [24] through DS records in each parent zone (Fig. 4.3). Signatures are checked over the canonical form of each RRset [15] for RSA/SHA-2, ECDSA and Ed25519 [25], [26], [27]. A complete chain is *secure*, a delegation without DS is *insecure*, and any failure is *bogus* and returned to clients as SERVFAIL.

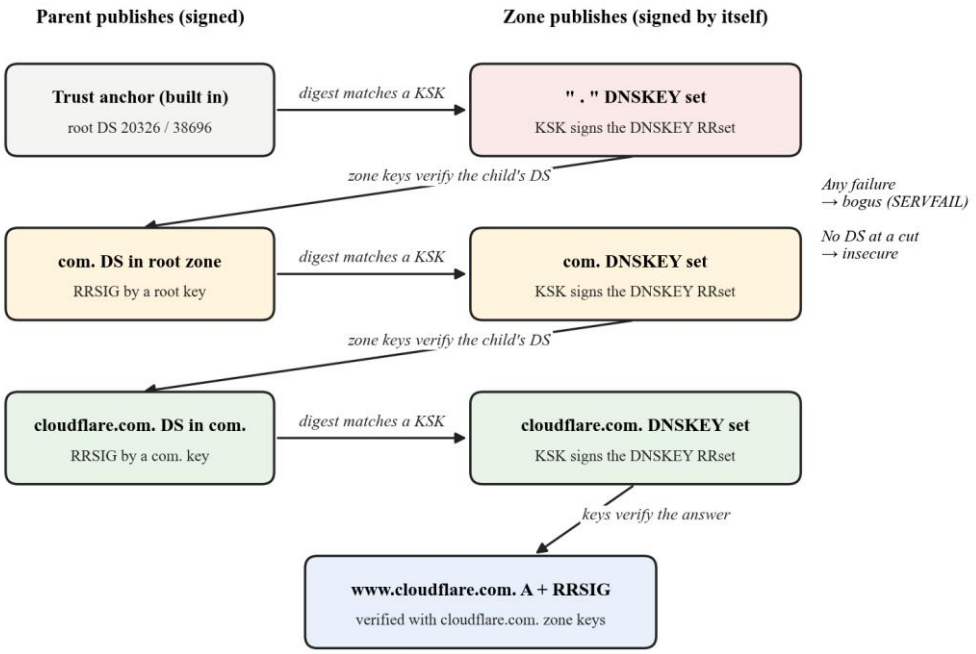


Fig. 4.3 Chain of Trust from the Root Trust Anchor to an Answer

Caching, server and testing

Answers, delegations and validated keys are cached with TTLs capped at one day and at most 10,000 entries per cache. In server mode (Fig. 4.4) all transports share one handler that refuses clients outside the access list, rate-limits each client (20 queries per second, burst 40), sets AD on secure answers, returns DNSSEC records only to clients that ask for them and truncates oversized UDP replies; DNS over TLS accepts TLS 1.3 only [28]. The test suite runs six real DNS servers on loopback addresses 127.0.0.10 to 127.0.0.15; three zones are DNSSEC-signed, and the servers include deliberately hostile behaviour such as poisoned glue, forged replies, altered signed records and a server that ignores letter case.

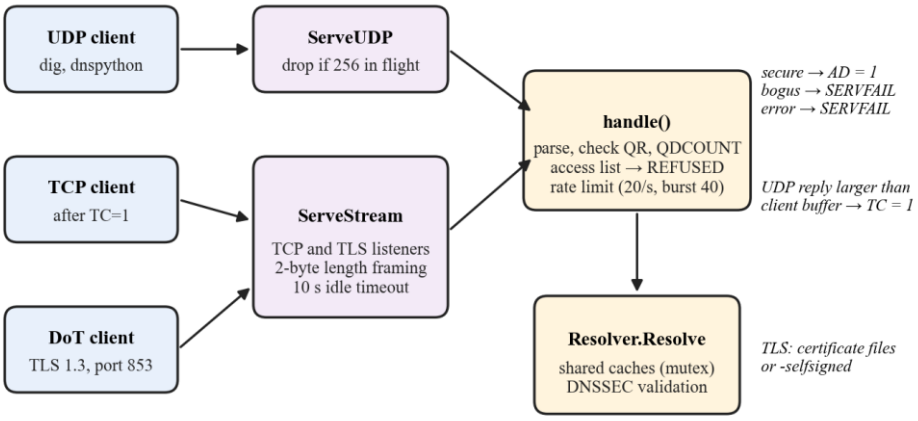


Fig. 4.4 Structure of the Recursive Server Mode

4.2 Codes and Standards

DNSR follows RFC 1034/1035 [1], [2], 2181 [6], 2308 [29], 5452 [5], 6891 [3], 7766 [10], the DNSSEC RFCs 4033 to 4035 [14], [15], [16] with the algorithm RFCs [25], [26], [27], RFC 9156 [17], RFC 7858 [18] and RFC 8446 [28]. The code is formatted with `gofmt`, checked with `go vet` and the race detector, and references follow the IEEE style.

4.3 Constraints, Alternatives and Trade-offs

Table 4.2 Main Design Decisions

Decision	Choice and reason
Language	Go [30]: memory-safe, concurrent, strong networking and cryptography libraries, single binary
Message handling	Hand-written codec instead of a library, as required, so the parser can be studied and hardened
Server selection	Random and sequential with a 2 s timeout: simple and polite, but slow when a server is down
DNSSEC scope	Positive answers only, within the semester; NSEC/NSEC3 proofs left as future work
Testing	Real servers on loopback: deterministic, offline and exercising the production code path

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The work was carried out over fourteen weeks, with tests written alongside each feature. Fig. 5.1 shows the schedule.

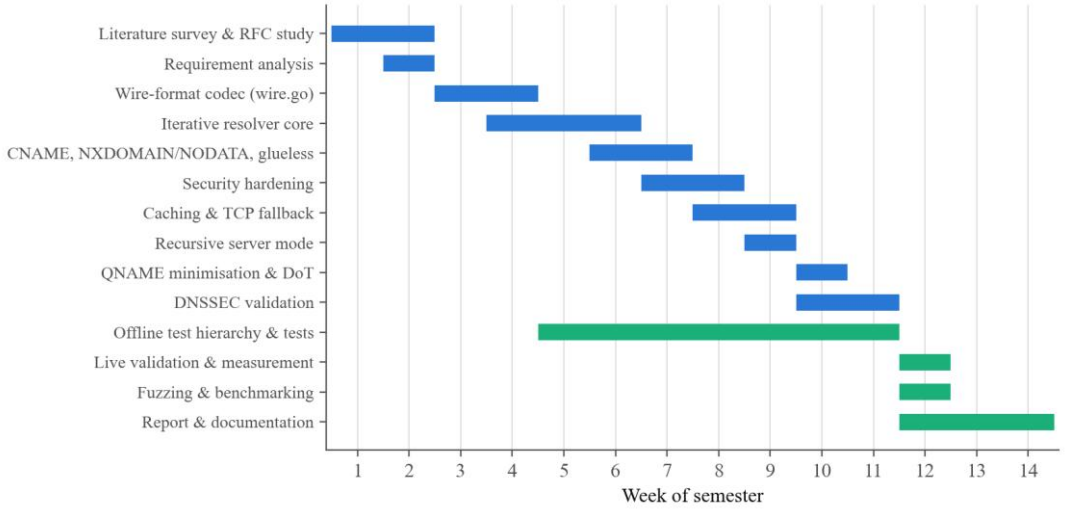


Fig. 5.1 Gantt Chart of the Project Schedule

Table 5.1 Milestones

Milestone	Week	Criterion
Codec complete	4	All record types parsed; compression test passes
Resolver feature-complete	8	Referral, CNAME, negative-answer, glueless and bailiwick tests pass
Caching, TCP and server	9	Cache, TCP and server tests pass
Security and privacy features	11	DNSSEC, minimisation, DoT, 0x20 and hardening tests pass
Validation and report	12 to 14	Live measurements, fuzzing, report submitted

CHAPTER 6

PROJECT DEMONSTRATION

6.1 Sample Codes

The listings are taken unchanged from the source. `readName` decodes compressed names with bounds checks and loop limits; `matchesQuery` and `randomCase` implement reply matching and 0x20; `glueFor` applies the glue bailiwick rule. The complete source is available in the project repository (Appendix A).

```
// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end, total := -1, 1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            if total += l + 1; total > 255 {
                return "", 0, errors.New("name longer than 255 bytes")
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
}
```

```
// matchesQuery is the anti-spoofing check: a reply is only accepted if it
// carries our ID and repeats exactly the question we asked.
func matchesQuery(m *Message, id uint16, name string, qtype RRType) bool {
    return m.Header.ID == id && m.Header.QR && len(m.Questions) == 1 &&
```

```
        strings.EqualFold(m.Questions[0].Name, name) && m.Questions[0].Type == qtype
}

// randomCase flips each letter of name to upper or lower case at random
// (the 0x20 technique), adding one bit of entropy per letter to the query.
func randomCase(name string) string {
    bits := make([]byte, len(name))
    crand.Read(bits)
    b := []byte(name)
    for i, c := range b {
        if 'a' <= c|0x20 && c|0x20 <= 'z' {
            b[i] = c&^0x20 | (bits[i]&1)<<5
        }
    }
    return string(b)
}

// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}
```

6.2 Sample Screen Shots

Fig. 6.1 shows a cold-cache lookup with QNAME minimisation: the root is asked only `com. NS`, a `.com` server only `github.com. NS`, and GitHub's server answers with a CNAME. Fig. 6.2 validates `cloudflare.com` with DNSSEC from the root key down to the answer.

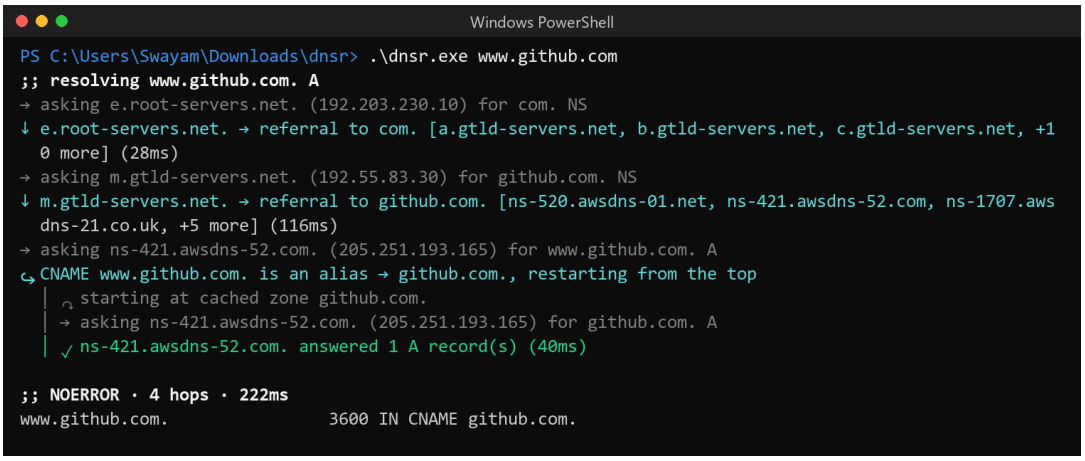


Fig. 6.1 Resolving www.github.com with QNAME Minimisation

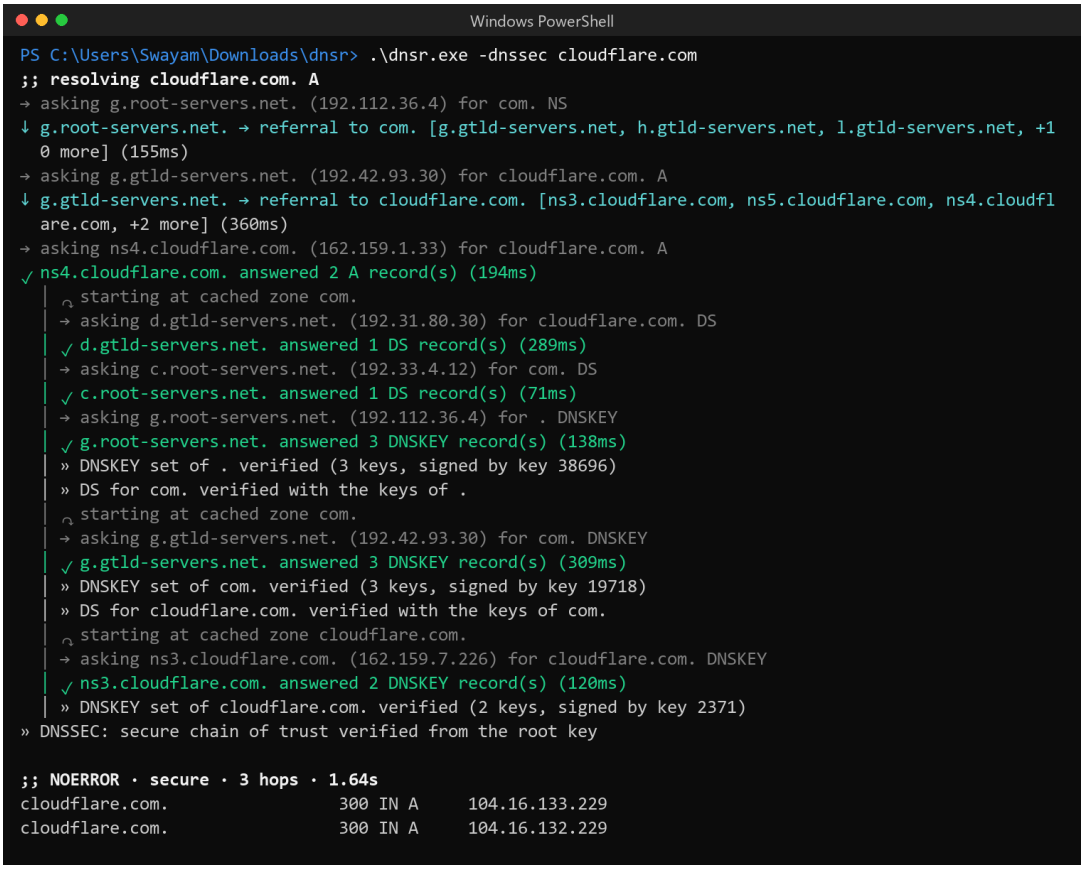


Fig. 6.2 DNSSEC Chain of Trust for cloudflare.com

Fig. 6.3 shows an independent client querying DNSR's server over UDP, TCP and DNS over TLS: secure answers carry AD, a deliberately mis-signed domain receives SERVFAIL, and a burst of 60 queries is partly rate-limited. Fig. 6.4 shows a client outside the access list being refused and a TLS 1.2 client being rejected.

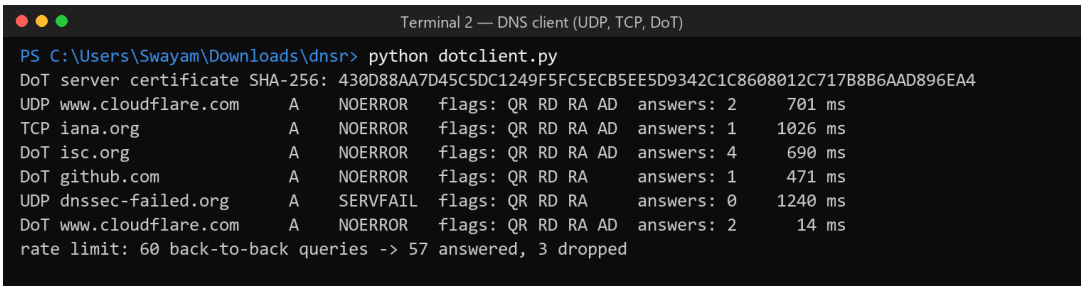


Fig. 6.3 An Independent Client Using UDP, TCP and DNS over TLS

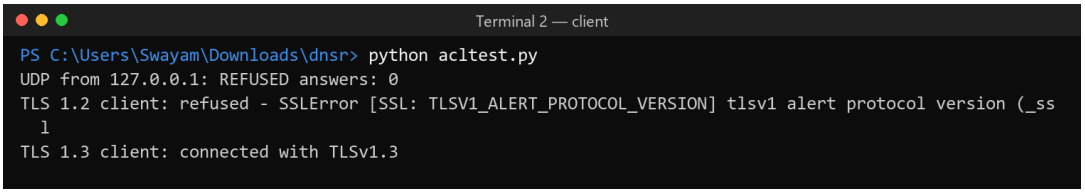


Fig. 6.4 Refused Client and TLS Version Enforcement

CHAPTER 7

RESULT AND DISCUSSION

7.1 Functional Tests

All 32 offline tests passed with `go test -v` and with the race detector (Table 7.1).

Table 7.1 Offline Test Results

Group	Tests
Resolution, CNAME, negative answers, MX, compression, TCP fallback	7
Poisoning, forged and mismatching replies	4
Randomness: IDs, 0x20 use and fall-back	3
Resource limits: TTL cap, cache bounds, expired delegation, glueless limit	4
QNAME minimisation (on and off)	2
DNSSEC: secure, tampered, wrong anchor, insecure, parsing	5
Server: AD and SERVFAIL, truncation, record stripping, rate limit, access list, DoT, TLS 1.3	7
Total: all PASS, 0 FAIL, 0 SKIP	32

7.2 Live Resolution and Caching

Fifteen popular domains were resolved three times each from empty caches on 20 October 2026. 44 of 45 runs succeeded (the other failed when four root servers timed out), all 15 domains returned the same response code as Google Public DNS, and 11 returned the same addresses; the rest are content-delivery networks that answer differently by location. The median cold-cache time was 557 ms (Fig. 7.1). The slow runs are caused by unresponsive servers and the two-second timeout. QNAME minimisation cost an extra query only for `vit.ac.in`, because `ac.in.` is not a separate zone.

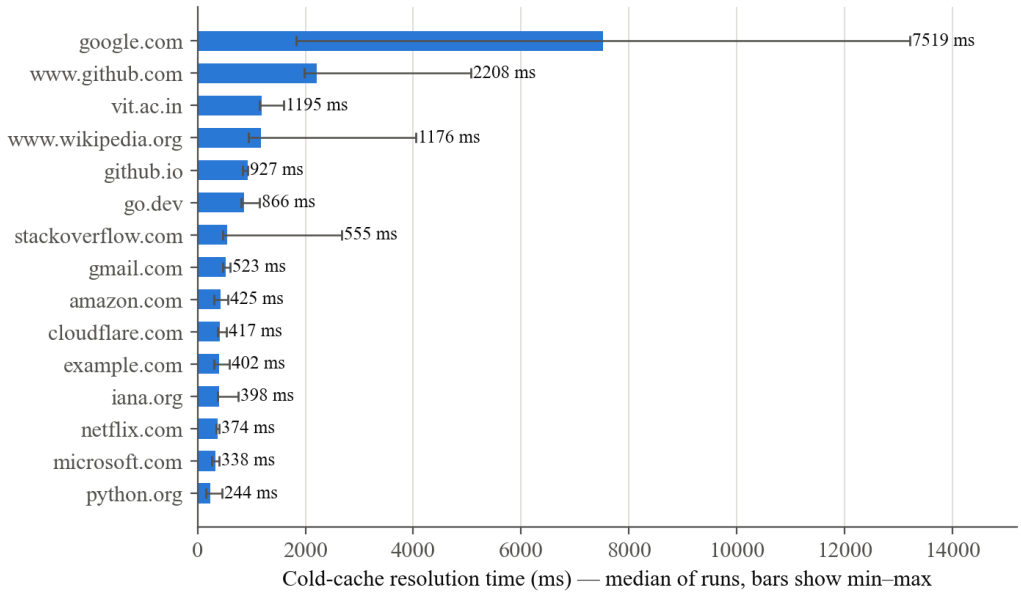


Fig. 7.1 Cold-Cache Resolution Time per Domain

Table 7.2 Effect of Caching (ms; reduction in brackets)

First name / sibling	Cold	Sibling (delegation cache)	Repeat (answer cache)
www.github.com / api.github.com	430	52 (88%)	98 (77%)
google.com / mail.google.com	721	134 (81%)	0 (100%)
www.wikipedia.org / en.wikipedia.org	769	128 (83%)	396 (49%)
vit.ac.in / www.vit.ac.in	1511	88 (94%)	0 (100%)
amazon.com / www.amazon.com	665	860 (-29%)	0 (100%)

The delegation cache made sibling names 81 to 94 percent faster in four of five cases, and repeated names were answered in under a millisecond, except aliases, whose CNAME is not cached, and `www.amazon.com`, which points into another provider's zone.

7.3 DNSSEC Validation

Table 7.3 DNSSEC Results (Median of Three Cold-Cache Runs)

Domain	Result	Plain (ms)	With -dnssec (ms)
cloudflare.com	secure	251	855
www.cloudflare.com	secure	208	1007
iana.org	secure	356	1026
isc.org	secure	406	981
www.ietf.org	secure	567	999
example.com	secure	281	1217

Domain	Result	Plain (ms)	With -dnssec (ms)
nic.cz	secure	355	722
github.com	insecure	280	477
google.com	insecure	314	533
vit.ac.in	insecure	3186	5102
dnssec-failed.org	bogus	2108	4230

All seven signed domains validated as secure, the three unsigned ones as insecure, and the deliberately mis-signed `dnssec-failed.org` as bogus, identically in all 66 validations. With empty caches validation added a median of 604 ms for fetching the DS and DNSKEY records along the chain; keys are cached per zone, so repeated queries are answered in milliseconds. The root zone was signed with the 2024 key during the tests, so both trust anchors were needed.

7.4 Server Mode, Security and Robustness

In server mode, repeated cached answers took 2 to 5 ms over UDP and about 10 ms over DNS over TLS; secure answers carried AD and the bogus domain received SERVFAIL. Of 100 back-to-back queries from one client, 92 were answered and the rest rate-limited. A client outside the access list received REFUSED and a TLS 1.2 client could not connect. Fig. 7.2 plots Eqn. 4.1: an attacker needs about 45,000 forged packets against the ID alone, 7.4×10^8 with the random port and about 6×10^{12} with 0x20; every live server preserved letter case, so 0x20 was always in effect.

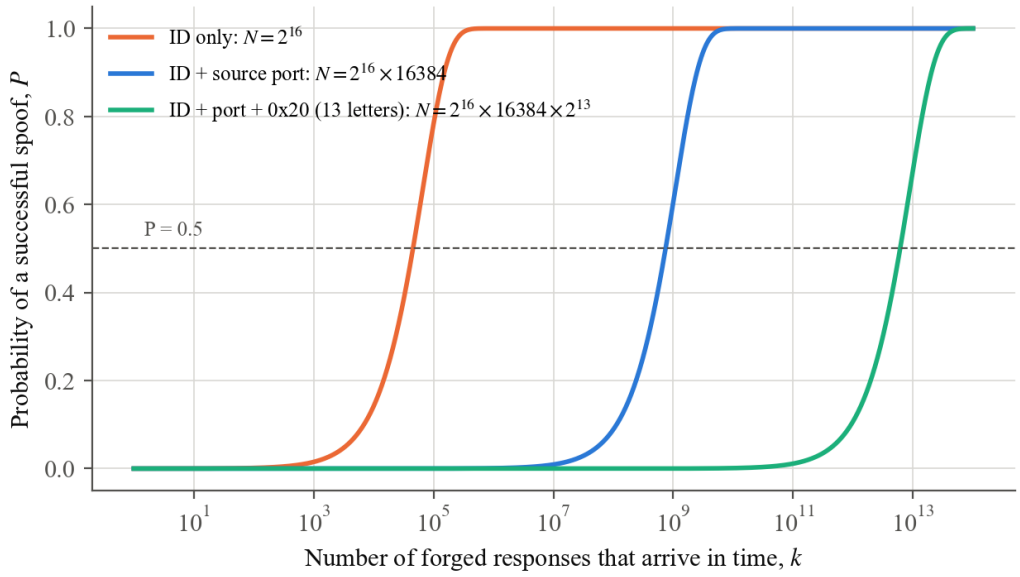


Fig. 7.2 Probability of a Successful Blind Spoof

Table 7.4 Robustness and Performance

Experiment	Result
1,000,000 random and mutated packets	835,319 rejected, 164,681 parsed, 0 panics, 0 hangs
Go coverage-guided fuzzing [31], 60 s	8,187,952 executions, no failure
Parse a reply / build a query	373.5 ns / 121.4 ns per operation
Verify an ECDSA P-256 signature	55.6 µs per RRset

7.5 Security Review and Goals

A review of the first version found nine weaknesses; eight were fixed (TTL cap, bounded caches, glueless limit, server limits with access list, TCP question check, 0x20, escaped trace output, default port) and one partly (DNSSEC, positive answers only). Table 7.5 summarises the goals.

Table 7.5 Achievement of Goals

ID	Result	Status
G1	15 of 15 domains match the public resolver; all record types parsed	Achieved
G2	Forged replies, poisoned glue and bad referrals rejected; spoof space about 10^{13}	Achieved
G3	7 secure, 3 insecure, 1 bogus in every run	Achieved (positive answers)
G4	Minimised upstream names; DNS over TLS 1.3	Achieved
G5	Sibling names 81 to 94% faster; repeats under 1 ms	Achieved
G6	Access list, rate limit, caps and TTL limit verified	Achieved
G7	About 9.2 million fuzz inputs, no failure	Achieved

7.6 Cost Analysis and Discussion

The project had no financial cost: all software is free and open source and the hardware was an existing laptop. The results show that about two thousand lines can resolve real names correctly while enforcing the checks that defeat cache poisoning and validating DNSSEC. Latency is dominated by unresponsive servers because of sequential server selection, aliases and negative answers are not fully cached, and DNSSEC does not yet prove that data is absent.

CHAPTER 8

SUMMARY

DNSR is a security-hardened iterative DNS resolver written from scratch in Go. It resolves names from the root with QNAME minimisation, defends against spoofing with random IDs, ports and letter case, strict reply matching and bailiwick checks, validates DNSSEC from the root trust anchors, and runs as a traced command-line tool or as an access-controlled, rate-limited server over UDP, TCP and DNS over TLS 1.3. It passed 32 offline tests, matched a public resolver for 15 live domains, validated every signed test domain and rejected a broken one, and survived about 9.2 million fuzzing inputs.

Recommended next steps are validation of NSEC and NSEC3 proofs, so that missing records and unsigned delegations can be proven; negative and CNAME caching; round-trip-time based server selection; DNS Cookies [32]; IPv6 transport; and DNS over HTTPS.

CHAPTER 9

REFERENCES

[1] P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, IETF, Nov. 1987.

[2] P. Mockapetris, "Domain Names – Implementation and Specification," RFC 1035, IETF, Nov. 1987.

[3] J. Damas, M. Graff, and P. Vixie, "Extension Mechanisms for DNS (EDNS(0))," RFC 6891, IETF, Apr. 2013.

[4] D. Kaminsky, "Black Ops 2008: It's the End of the Cache as We Know It," presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2008.

[5] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," RFC 5452, IETF, Jan. 2009.

[6] R. Elz and R. Bush, "Clarifications to the DNS Specification," RFC 2181, IETF, Jul. 1997.

[7] S. Son and V. Shmatikov, "The Hitchhiker's Guide to DNS Cache Poisoning," in Proc. SecureComm 2010, LNICST, vol. 50, Springer, 2010, pp. 466–483.

[8] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, "DNS Cache Poisoning Attack Reloaded: Revolutions with Side Channels," in Proc. ACM CCS '20, 2020, pp. 1337–1350.

[9] DNS Flag Day 2020. [Online]. Available: https://dnsflagday.net/2020/

[10] J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, and D. Wessels, "DNS Transport over TCP – Implementation Requirements," RFC 7766, IETF, Mar. 2016.

[11] D. Dagon, M. Antonakakis, P. Vixie, T. Jinmei, and W. Lee, "Increased DNS Forgery Resistance Through 0x20-Bit Encoding," in Proc. ACM CCS '08, 2008, pp. 211–222.

[12] Y. Afek, A. Bremler-Barr, and L. Shafir, "NXNSAttack: Recursive DNS Inefficiencies and Vulnerabilities," in Proc. 29th USENIX Security Symposium, 2020, pp. 631–648.

[13] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, "DNS Performance and the Effectiveness of Caching," IEEE/ACM Trans. Networking, vol. 10, no. 5, pp. 589–603, Oct. 2002.

[14] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, Mar. 2005.

[15] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Resource Records for the DNS Security Extensions," RFC 4034, IETF, Mar. 2005.

[16] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Protocol Modifications for the DNS Security Extensions," RFC 4035, IETF, Mar. 2005.

[17] S. Bortzmeyer, R. Dolmans, and P. Hoffman, "DNS Query Name Minimisation to Improve Privacy," RFC 9156, IETF, Nov. 2021.

[18] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over Transport Layer Security (TLS)," RFC 7858, IETF, May 2016.

[19] Internet Systems Consortium, "BIND 9." [Online]. Available: https://www.isc.org/bind/

[20] NLnet Labs, "Unbound – A Validating, Recursive, Caching DNS Resolver." [Online]. Available: https://nlnetlabs.nl/projects/unbound/

[21] M. Gieben, "dns – A DNS Library in Go." [Online]. Available: https://github.com/miekg/dns

[22] B. Halley et al., "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org/

[23] Microsoft, "The default dynamic port range for TCP/IP has changed since Windows Vista and in Windows Server 2008," Microsoft Learn, KB929851. [Online]. Available: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/default-dynamic-port-range-tcpip-chang

[24] IANA, "DNSSEC Trust Anchors (root-anchors.xml)." [Online]. Available: https://data.iana.org/root-anchors/root-anchors.xml

[25] J. Jansen, "Use of SHA-2 Algorithms with RSA in DNSKEY and RRSIG Resource Records for DNSSEC," RFC 5702, IETF, Oct. 2009.

[26] P. Hoffman and W. C. A. Wijngaards, "Elliptic Curve Digital Signature Algorithm (DSA) for DNSSEC," RFC 6605, IETF, Apr. 2012.

[27] O. Sury and R. Edmonds, "Edwards-Curve Digital Security Algorithm (EdDSA) for DNSSEC," RFC 8080, IETF, Feb. 2017.

[28] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018.

[29] M. Andrews, "Negative Caching of DNS Queries (DNS NCACHE)," RFC 2308, IETF, Mar. 1998.

[30] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[31] The Go Authors, "Go Fuzzing." [Online]. Available: https://go.dev/doc/security/fuzz/

[32] D. Eastlake 3rd and M. Andrews, "Domain Name System (DNS) Cookies," RFC 7873, IETF, May 2016.

APPENDIX A

SOURCE CODE AND USAGE

The complete source code, tests and diagrams are in the project repository at https://github.com/swayam5342/dnsr (branch dnssec-hardening). The full listing is also given in Appendix A of the extended version of this report.

```
go build -o dnsr .                    # requires Go 1.25 or later
go test -v ./...                      # 32 offline tests
./dnsr -dnssec www.github.com           # resolve with DNSSEC validation
./dnsr serve -dnssec -dot 127.0.0.1:8853 -selfsigned
./dnsr serve -addr 0.0.0.0:8053 -allow 127.0.0.0/8,192.168.0.0/16
```